

Objectives

- Demonstrate creation of Spring Boot Application
 - Spring initializr, <https://start.spring.io>, @SpringBootApplication, SpringApplication.run()
 - Ref - <https://start.spring.io>
- Explain the need and benefits of Spring Boot
 - Makes Java development easy, avoids tedious development steps, reduces development time, avoids writing boilerplate code, provides embedded tomcat server, avoid XML configuration
 - Ref - <https://www.journaldev.com/7969/spring-boot-tutorial>
- Demonstrate loading bean from spring configuration file
 - Spring configuration xml, spring xml schema spring-beans.xsd, <bean>, id, class, <constructor-arg>, <property>, name, value, ClassPathXmlApplicationContext, ApplicationContext, context.getBean(), singleton scope, prototype scope
 - Ref - <https://docs.spring.io/spring-framework/docs/current/spring-framework-reference/core.html>
 - IoC Container - <https://docs.spring.io/spring-framework/docs/current/spring-framework-reference/core.html#beans>
 - Scopes - <https://docs.spring.io/spring-framework/docs/current/spring-framework-reference/core.html#beans-factory-scopes>
 - Constructor Injection - <https://docs.spring.io/spring-framework/docs/current/spring-framework-reference/core.html#beans-constructor-injection>
 - Setter method injection - <https://docs.spring.io/spring-framework/docs/current/spring-framework-reference/core.html#beans-setter-injection>
- Demonstrate inclusion of logging in Spring Boot Application
 - application.properties, logging.level, logging.pattern, server.port, LoggerFactory, Logger, log levels (trace, debug, info, warn, error)
 - Ref - <https://docs.spring.io/spring-boot/docs/current/reference/html/boot-features-logging.html>

Hands on 1

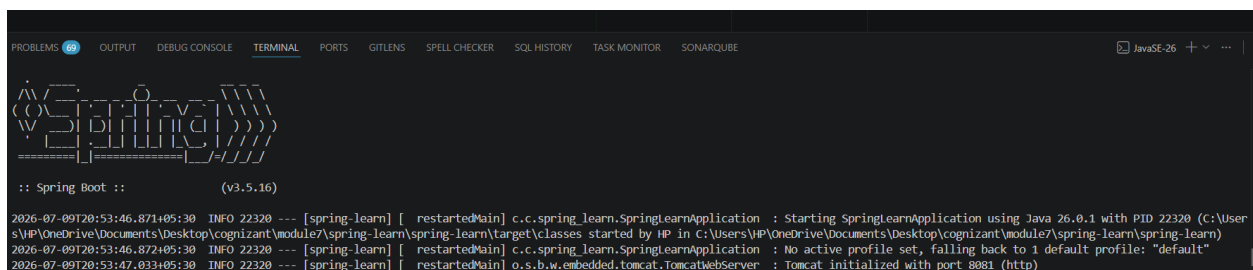
Create a Spring Web Project using Maven

Follow steps below to create a project:

1. Go to <https://start.spring.io/>
2. Change Group as "com.cognizant"
3. Change Artifact Id as "spring-learn"
4. Select Spring Boot DevTools and Spring Web
5. Create and download the project as zip
6. Extract the zip in root folder to Eclipse Workspace
7. Build the project using 'mvn clean package -Dhttp.proxyHost=proxy.cognizant.com -Dhttp.proxyPort=6050 -Dhttps.proxyHost=proxy.cognizant.com -Dhttps.proxyPort=6050 -Dhttp.proxyUser=123456' command in command line
8. Import the project in Eclipse "File > Import > Maven > Existing Maven Projects > Click Browse and select extracted folder > Finish"
9. Include logs to verify if main() method of SpringLearnApplication.
10. Run the SpringLearnApplication class.

SME to walk through the following aspects related to the project created:

1. src/main/java - Folder with application code
2. src/main/resources - Folder for application configuration
3. src/test/java - Folder with code for testing the application
4. SpringLearnApplication.java - Walkthrough the main() method.
5. Purpose of @SpringBootApplication annotation
6. pom.xml
 1. Walkthrough all the configuration defined in XML file
 2. Open 'Dependency Hierarchy' and show the dependency tree.



The screenshot shows the Eclipse IDE interface with the 'TERMINAL' tab active. The terminal displays the Spring Boot logo and logs for a Spring Boot application (v3.5.16). The logs indicate that the application was restarted using Java 26.0.1 with PID 22320. The logs also show that the application is running on port 8081 (http).

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GIT LENS SPELL CHECKER SQL HISTORY TASK MONITOR SONARQUBE
:: Spring Boot :: (v3.5.16)
2026-07-09T20:53:46.871+05:30 INFO 22320 --- [spring-learn] [ restartedMain] c.c.spring_learn.SpringLearnApplication : Starting SpringLearnApplication using Java 26.0.1 with PID 22320 (C:\User
s\HP\OneDrive\Documents\Desktop\cognizant\module7\spring-learn\spring-learn\target\classes started by HP in C:\Users\HP\OneDrive\Documents\Desktop\cognizant\module7\spring-learn\spring-learn)
2026-07-09T20:53:46.872+05:30 INFO 22320 --- [spring-learn] [ restartedMain] c.c.spring_learn.SpringLearnApplication : No active profile set, falling back to 1 default profile: "default"
2026-07-09T20:53:47.033+05:30 INFO 22320 --- [spring-learn] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8081 (http)
```

Spring Core – Load SimpleDateFormat from Spring Configuration XML

SimpleDateFormat with the pattern 'dd/MM/yyyy' is created in multiple places of an application. To avoid creation of SimpleDateFormat in multiple places, define a bean in Spring XML Configuration file and retrieve the date.

Follow steps below to implement:

- Create spring configuration file **date-format.xml** in **src/main/resources** folder of 'spring-learn' project
- Open <https://docs.spring.io/spring-framework/docs/current/spring-framework-reference/core.html#beans-factory-metadata>
- Copy the XML defined in the section of previous step URL and paste it into **date-format.xml**
- Define bean tag in the XML with for date format. Refer code below.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="dateFormat" class="java.text.SimpleDateFormat">
        <constructor-arg value="dd/MM/yyyy" />
    </bean>

</beans>
```

- Create new method **displayDate()** in **SpringLearnApplication.java**
- In displayDate() method create the **ApplicationContext**. Refer code below:

```
ApplicationContext context = new ClassPathXmlApplicationContext("date-format.xml");
```

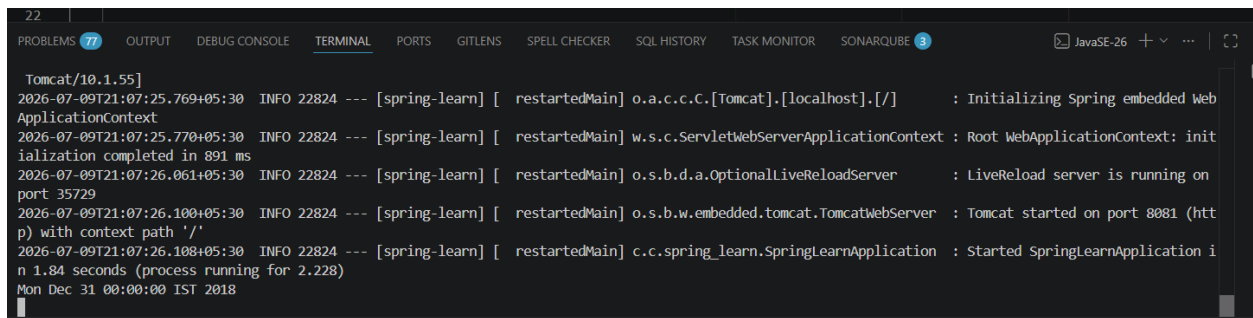
- Get the `dateFormat` using `getBean()` method. Refer code below.

```
SimpleDateFormat format = context.getBean("dateFormat", SimpleDateFormat.class);
```

- Using the format variable try to parse string '31/12/2018' to `Date` class and display the result using `System.out.println`.
- Run the application as 'Java Application' and check the result in console log output.

Troubleshooting Tips

If the tomcat port has a conflict and the server is not starting include the below property in `application.properties` file in `src/main/resources` folder.



```
Tomcat/10.1.55]
2026-07-09T21:07:25.769+05:30 INFO 22824 --- [spring-learn] [ restartedMain] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded Web
ApplicationContext
2026-07-09T21:07:25.770+05:30 INFO 22824 --- [spring-learn] [ restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: init
ialization completed in 891 ms
2026-07-09T21:07:26.061+05:30 INFO 22824 --- [spring-learn] [ restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on
port 35729
2026-07-09T21:07:26.100+05:30 INFO 22824 --- [spring-learn] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8081 (htt
p) with context path '/'
2026-07-09T21:07:26.108+05:30 INFO 22824 --- [spring-learn] [ restartedMain] c.c.spring_learn.SpringLearnApplication : Started SpringLearnApplication i
n 1.84 seconds (process running for 2.228)
Mon Dec 31 00:00:00 IST 2018
```

Hands on 3

Spring Core - Incorporate Logging

Incorporate logging in the Spring Boot project created in previous hands on. Refer steps below:

- Create `application.properties` if not yet created in `src/main/resources` folder
- Add below lines in `application.properties`

```
logging.level.org.springframework=info
logging.level.com.cognizant.springlearn=debug
logging.pattern.console=%d{yyMMdd} | %d{HH:mm:ss.SSS} | %-20.20thread | %5p | %-25.25logger{25} | %25M | %m%n
```

- In `SpringLearnApplication.java` include the following imports:

```
import org.slf4j.Logger;

import org.slf4j.LoggerFactory;
```

- Include the below static variable in **SpringLearnApplication.java**:

```
private static final Logger LOGGER = LoggerFactory.getLogger(SpringLearnApplica
tion.class);
```

- Include info log on start and end of method. Debug log for displaying the date (refer code below)

```
public void displayDate() {

    LOGGER.info("START");

    //..

    LOGGER.debug(date);

    //..

    LOGGER.info("END");

}
```

IMPORTANT NOTE: Going forward all methods should incorporate logging as specified above. **Never** use `System.out.println()`.

```
260709|21:30:55.642|restartedMain|INFO|s.b.w.e.t.TomcatWebServer|initialize|Tomcat initialized with port 8081 (http)
260709|21:30:55.673|restartedMain|INFO|o.a.c.c.StandardService|log|Starting service [Tomcat]
260709|21:30:55.674|restartedMain|INFO|o.a.c.c.core.StandardEngine|log|Starting Servlet engine: [Apache Tomcat/10.1.55]
260709|21:30:55.769|restartedMain|INFO|o.a.c.c.C.[.]/|log|Initializing Spring embedded WebApplicationContext
260709|21:30:55.770|restartedMain|INFO|bServerApplicationContext|prepareWebApplicationContext|Root WebApplicationContext: initialization completed in 1094 ms
260709|21:30:56.023|restartedMain|INFO|.OptionalLiveReloadServer|startServer|LiveReload server is running on port 35729
260709|21:30:56.073|restartedMain|INFO|s.b.w.e.t.TomcatWebServer|start|Tomcat started on port 8081 (http) with context path '/'
260709|21:30:56.079|restartedMain|INFO|.s.SpringLearnApplication|logStarted|Started SpringLearnApplication in 1.823 seconds (process running for 2.107)
260709|21:30:56.083|restartedMain|INFO|.s.SpringLearnApplication|displayDate|START
260709|21:30:56.161|restartedMain|DEBUG|.s.SpringLearnApplication|displayDate|Date : Mon Dec 31 00:00:00 IST 2018
260709|21:30:56.163|restartedMain|INFO|.s.SpringLearnApplication|displayDate|END
```

Hands on 4

Spring Core – Load Country from Spring Configuration XML

An airlines website is going to support booking on four countries. There will be a drop down on the home page of this website to select the respective country. It is also important to store the two-character ISO code of each country.

Code	Name
US	United States
DE	Germany
IN	India
JP	Japan

Above data has to be stored in spring configuration file. Write a program to read this configuration file and display the details.

Steps to implement

- Pick any one of your choice country to configure in Spring XML configuration named **country.xml**.
- Create a bean tag in spring configuration for country and set the property and values

```
<bean id="country" class="com.cognizant.springlearn.Country">
    <property name="code" value="IN" />
    <property name="name" value="India" />
</bean>
```

- Create **Country** class with following aspects:
 - Instance variables for **code** and **name**
 - Implement empty parameter constructor with inclusion of debug log within the constructor with log message as **"Inside Country Constructor."**
 - Generate getters and setters with inclusion of debug with relevant message within each setter and getter method.
 - Generate **toString()** method
- Create a method **displayCountry()** in **SpringLearnApplication.java**, which will read the country bean from spring configuration file and display the country details. **ClassPathXmlApplicationContext**, **ApplicationContext** and **context.getBean("beanId", Country.class)**. Refer sample code for displayCountry() method below.

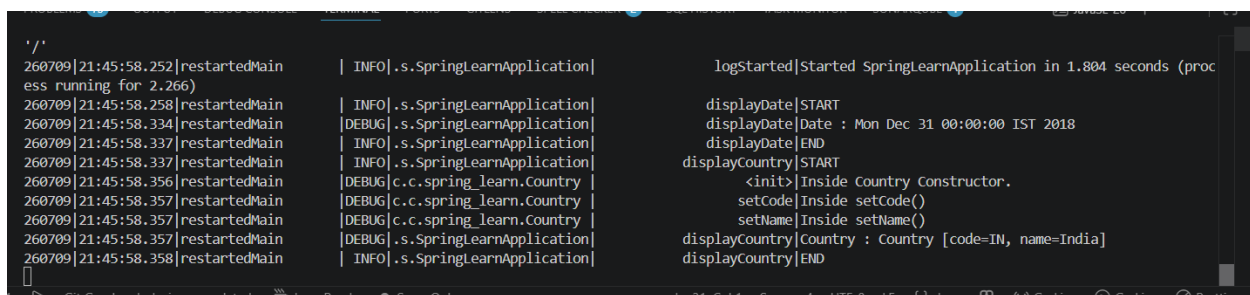
```
ApplicationContext context = new ClassPathXmlApplicationContext("country.xml");
Country country = (Country) context.getBean("country", Country.class);
```

```
LOGGER.debug("Country : {}", country.toString());
```

- Invoke `displayCountry()` method in `main()` method of `SpringLearnApplication.java`.
- Execute `main()` method and check the logs to find out which constructors and methods were invoked.

SME to provide more detailing about the following aspects:

- bean tag, id attribute, class attribute, property tag, name attribute, value attribute
- `ApplicationContext`, `ClassPathXmlApplicationContext`
- What exactly happens when `context.getBean()` is invoked



```
260709|21:45:58.252|restartedMain | INFO|.s.SpringLearnApplication| logStarted|Started SpringLearnApplication in 1.804 seconds (proc  
ess running for 2.266)  
260709|21:45:58.258|restartedMain | INFO|.s.SpringLearnApplication| displayDate|START  
260709|21:45:58.334|restartedMain | DEBUG|.s.SpringLearnApplication| displayDate|Date : Mon Dec 31 00:00:00 IST 2018  
260709|21:45:58.337|restartedMain | INFO|.s.SpringLearnApplication| displayDate|END  
260709|21:45:58.337|restartedMain | INFO|.s.SpringLearnApplication| displayCountry|START  
260709|21:45:58.356|restartedMain | DEBUG|c.c.spring_learn.Country | <init>|Inside Country Constructor.  
260709|21:45:58.357|restartedMain | DEBUG|c.c.spring_learn.Country | setCode|Inside setCode()  
260709|21:45:58.357|restartedMain | DEBUG|c.c.spring_learn.Country | setName|Inside setName()  
260709|21:45:58.357|restartedMain | DEBUG|.s.SpringLearnApplication| displayCountry|Country : Country [code=IN, name=India]  
260709|21:45:58.358|restartedMain | INFO|.s.SpringLearnApplication| displayCountry|END
```

Hands on 5

Spring Core – Demonstration of Singleton Scope and Prototype Scope

The Country bean done in the previous hands on will be used to demonstrate the scopes in Spring. Implement the steps below.

Follow steps below to demonstrate Singleton Scope

- Include a line in `displayCountry()` to get country bean reference one more time from the same application context. Only the third line of the below code snippet should be copied and pasted.

```
ApplicationContext context = new ClassPathXmlApplicationContext("country.xml");  
Country country = context.getBean("country", Country.class);  
Country anotherCountry = context.getBean("country", Country.class);
```

- The constructor will be called only once, which means that only one instance of Country bean is created

Follow steps below to demonstrate Prototype Scope

- Include scope="prototype" attribute in bean definition xml.

```
<bean id="country" class="com.cognizant.springlearn.Country" scope="prototype">
```

- Run the application
- Constructor will be called twice, which means that two instances of country is created.

```
260709|21:53:39.079|restartedMain|DEBUG|.s.SpringLearnApplication|displayDate|Date : Mon Dec 31 00:00:00 IST 2018
260709|21:53:39.080|restartedMain|INFO|.s.SpringLearnApplication|displayDate|END
260709|21:53:39.080|restartedMain|INFO|.s.SpringLearnApplication|displayCountry|START
260709|21:53:39.097|restartedMain|DEBUG|c.c.spring_learn.Country|<init>|Inside Country Constructor.
260709|21:53:39.098|restartedMain|DEBUG|c.c.spring_learn.Country|setCode|Inside setCode()
260709|21:53:39.098|restartedMain|DEBUG|c.c.spring_learn.Country|setName|Inside setName()
260709|21:53:39.099|restartedMain|DEBUG|c.c.spring_learn.Country|<init>|Inside Country Constructor.
260709|21:53:39.099|restartedMain|DEBUG|c.c.spring_learn.Country|setCode|Inside setCode()
260709|21:53:39.099|restartedMain|DEBUG|c.c.spring_learn.Country|setName|Inside setName()
260709|21:53:39.099|restartedMain|DEBUG|.s.SpringLearnApplication|displayCountry|Country : Country [code=IN, name=India]
260709|21:53:39.100|restartedMain|DEBUG|.s.SpringLearnApplication|displayCountry|Another Country : Country [code=IN, name=India]
260709|21:53:39.100|restartedMain|INFO|.s.SpringLearnApplication|displayCountry|END
```

Hands on 6

Spring Core – Load list of countries from Spring Configuration XML

Our main objective was to retrieve the list of four countries for the airlines website. Refer steps below to get this incorporated.

- Create a separate bean for each of the four country in country.xml.
- Create an ArrayList of Country in country.xml. Refer code below.

```
<bean id="countryList" class="java.util.ArrayList">
    <constructor-arg>
        <list>
            <ref bean="in"></ref>
            <ref bean="us"></ref>
            <ref bean="de"></ref>
```

```

        <ref bean="jp"></ref>

    </list>

</constructor-arg>

</bean>

```

- Include new method displayCountries() in SpringLearnApplication.java
- In displayCountries() read the country list created above
- Display the list of countries as debug log.

SME to provide detailing on below aspects:

- <list>
- <ref>
- bean attribute

IMPORTANT NOTE: Do not forget to include the start and end logs in this new method.

```

260709|22:05:26.379|restartedMain|DEBUG|c.c.spring_learn.Country|<init>|Inside Country Constructor.
260709|22:05:26.379|restartedMain|DEBUG|c.c.spring_learn.Country|setCode|Inside setCode()
260709|22:05:26.379|restartedMain|DEBUG|c.c.spring_learn.Country|setName|Inside setName()
260709|22:05:26.379|restartedMain|DEBUG|c.c.spring_learn.Country|<init>|Inside Country Constructor.
260709|22:05:26.380|restartedMain|DEBUG|c.c.spring_learn.Country|setCode|Inside setCode()
260709|22:05:26.380|restartedMain|DEBUG|c.c.spring_learn.Country|setName|Inside setName()
260709|22:05:26.380|restartedMain|DEBUG|c.c.spring_learn.Country|<init>|Inside Country constructor.
260709|22:05:26.380|restartedMain|DEBUG|c.c.spring_learn.Country|setCode|Inside setCode()
260709|22:05:26.380|restartedMain|DEBUG|c.c.spring_learn.Country|setName|Inside setName()
260709|22:05:26.385|restartedMain|DEBUG|.s.SpringLearnApplication|displayCountries|Countries : [Country [code=IN, name=India], Country [code=US, name=United States], Country [code=DE, name=Germany], Country [code=JP, name=Japan]]
260709|22:05:26.385|restartedMain|INFO|.s.SpringLearnApplication|displayCountries|END

```